

UNIVERSIDADE DO VALE DO ITAJAÍ – UNIVALI
ESCOLA POLITÉCNICA

THIAGO YUKIO HORITA PACHECO

RELATÓRIO PROJETO M1
OPERAÇÕES PONTUAIS E FILTRAGEM ESPACIAL

ITAJAÍ
2026

SUMÁRIO

1. ENUNCIADO E OBJETIVOS DO PROJETO
2. EXPLICAÇÃO E CONTEXTO DA APLICAÇÃO
3. DESENVOLVIMENTO
 - 3.1 Manipulação de pixels
 - 3.2 Manipulação de bits
 - 3.3 Questões diversas
 - 3.4 Transformação de histograma
 - 3.5 Detecção de bordas
 - 3.6 Filtragem High-Boost
 - 3.7 Filtragem Bilateral
4. CÓDIGOS IMPORTANTES DA IMPLEMENTAÇÃO
5. ANÁLISE E DISCUSSÃO SOBRE OS RESULTADOS FINAIS
7. CONSIDERAÇÕES FINAIS

1. ENUNCIADO E OBJETIVOS DO PROJETO

O Projeto M1 da disciplina de Processamento de Imagens tem como objetivo consolidar conceitos fundamentais de reconhecimento e processamento digital de imagens por meio de implementações em Python. O roteiro envolve manipulação de pixels, manipulação de bits, questões sobre representação e transmissão, transformação de histogramas, detecção de bordas e filtragens High-Boost e Bilateral.

O notebook analisado foi desenvolvido em Python, utilizando OpenCV, NumPy e Matplotlib. O trabalho parte das imagens do conjunto da disciplina e constrói funções próprias para várias operações, principalmente quando a atividade exige implementação procedural.

Este relatório documenta o conteúdo efetivamente presente no arquivo TrabalhoM1_ThiagoPacheco.ipynb, organizando a implementação, os parâmetros, os experimentos e os resultados registrados. Etapas ausentes ou incompletas no notebook são identificadas explicitamente.

1.1 Organização

- Manipulação de pixels e composição de imagens.
- Quantização e planos de bits.
- Questões sobre MSB, LSB e transmissão.
- Equalização e especificação de histogramas.
- Detecção de bordas com Canny, Roberts, Prewitt, Sobel e Laplaciano.
- Filtragem High-Boost com kernels Gaussianos.
- Filtragem Bilateral e busca de parâmetros por distância L2.

2. EXPLICAÇÃO E CONTEXTO DA APLICAÇÃO

O processamento digital de imagens representa uma imagem como uma matriz de pixels. O projeto explora diretamente essa representação, permitindo observar como operações matemáticas sobre intensidades e componentes de cor produzem alterações visuais.

Na manipulação de pixels, a cor predominante de uma imagem é usada como referência para construir uma máscara e substituir partes do primeiro plano por pixels de uma imagem de fundo. A tolerância controla o quanto a cor pode variar e ainda ser considerada pertencente à região a substituir.

Na manipulação de bits, cada pixel de 8 bits é analisado por seus planos binários. A quantização reduz a quantidade de níveis de intensidade disponíveis, enquanto a extração dos planos permite visualizar a contribuição de cada posição binária.

Na transformação de histogramas, o trabalho utiliza a distribuição das intensidades para realizar equalização e construir uma especificação entre duas imagens. Na detecção de bordas, diferentes operadores evidenciam transições de intensidade. Finalmente, High-Boost e Bilateral possuem objetivos distintos: realce de altas frequências e suavização com preservação de bordas, respectivamente.

2.1 Ambiente

O notebook inicia com cv2, NumPy, Matplotlib e cv2_imshow, e utiliza caminhos do Google Drive, indicando execução em Google Colab.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab.patches import cv2_imshow
```


3. DESENVOLVIMENTO

3.1 Manipulação de pixels

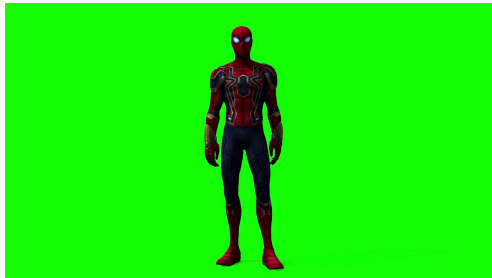
A função `get_most_common_color` achata a imagem em uma matriz de pixels e utiliza `np.unique` com `axis=0` para encontrar as combinações de cores e suas frequências. O índice da maior contagem identifica a cor predominante.

```
def get_most_common_color(img):  
    pixels = img.reshape(-1, img.shape[-1])  
    colors, counts = np.unique(pixels, axis=0, return_counts=True)  
    most_common_bgr = colors[counts.argmax()]  
    most_common_rgb = most_common_bgr[::-1]  
    return most_common_rgb
```

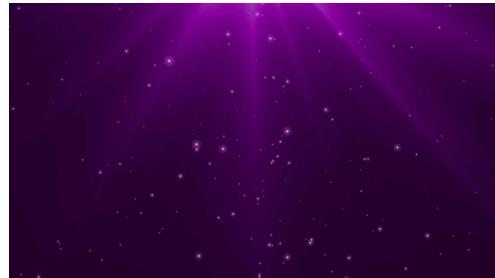
A função `mergeImage` redimensiona o fundo para as dimensões do primeiro plano, obtém a cor predominante e cria uma máscara. Com `tolerance=0`, a comparação é exata; com tolerância positiva, cada componente é comparado por diferença absoluta.

```
def mergeImage(img, bg, tolerance=0):  
    bg_resized = cv2.resize(bg, (img.shape[1], img.shape[0]))  
    most_common_color = get_most_common_color(img)  
    if tolerance == 0:  
        mask = np.all(img == most_common_color, axis=-1)  
    else:  
        diff = np.abs(img.astype(int) - most_common_color.astype(int))  
        mask = np.all(diff <= tolerance, axis=-1)  
    result = img.copy()  
    result[mask] = bg_resized[mask]  
    return cv2_imshow(result)
```

Foram testadas quatro combinações de primeiro plano e plano de fundo, usando tolerâncias 120 e 100.



fg.jpg



bg.jpg



Resultado de mergelImage com tolerância 120



fg1.jpg



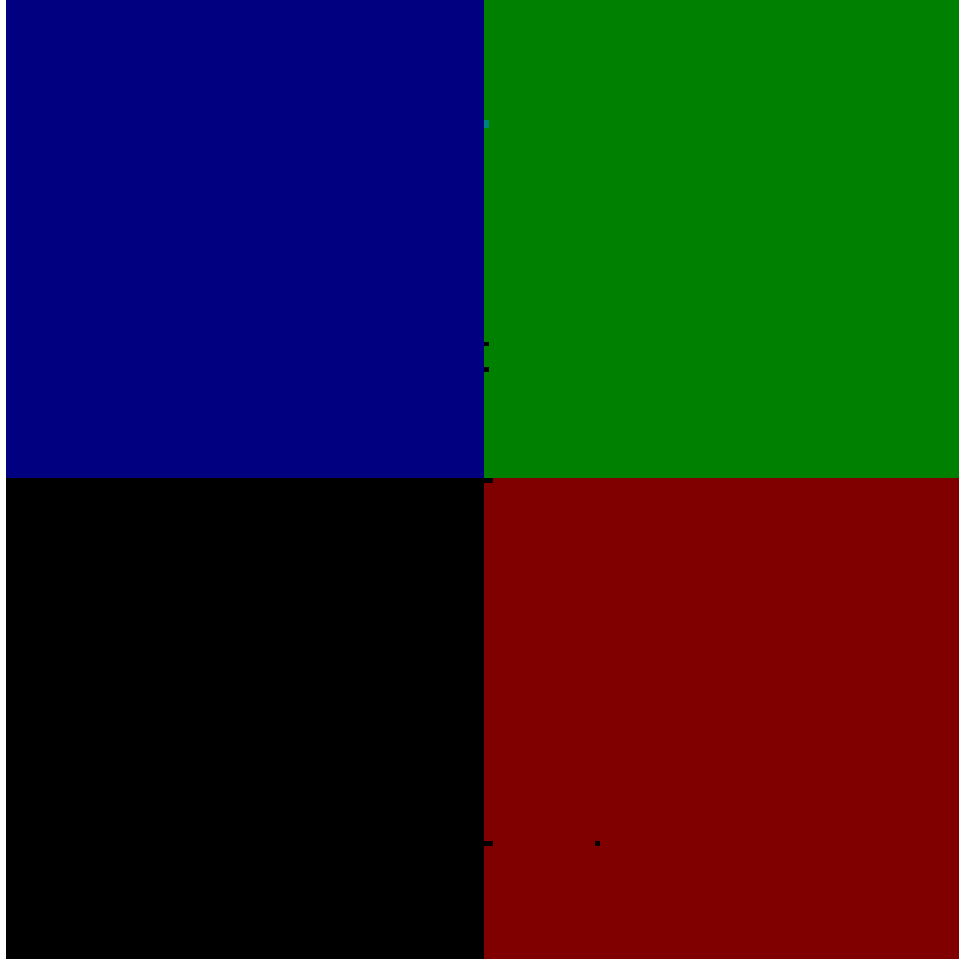
bg1.jpg

3.2 Manipulação de bits

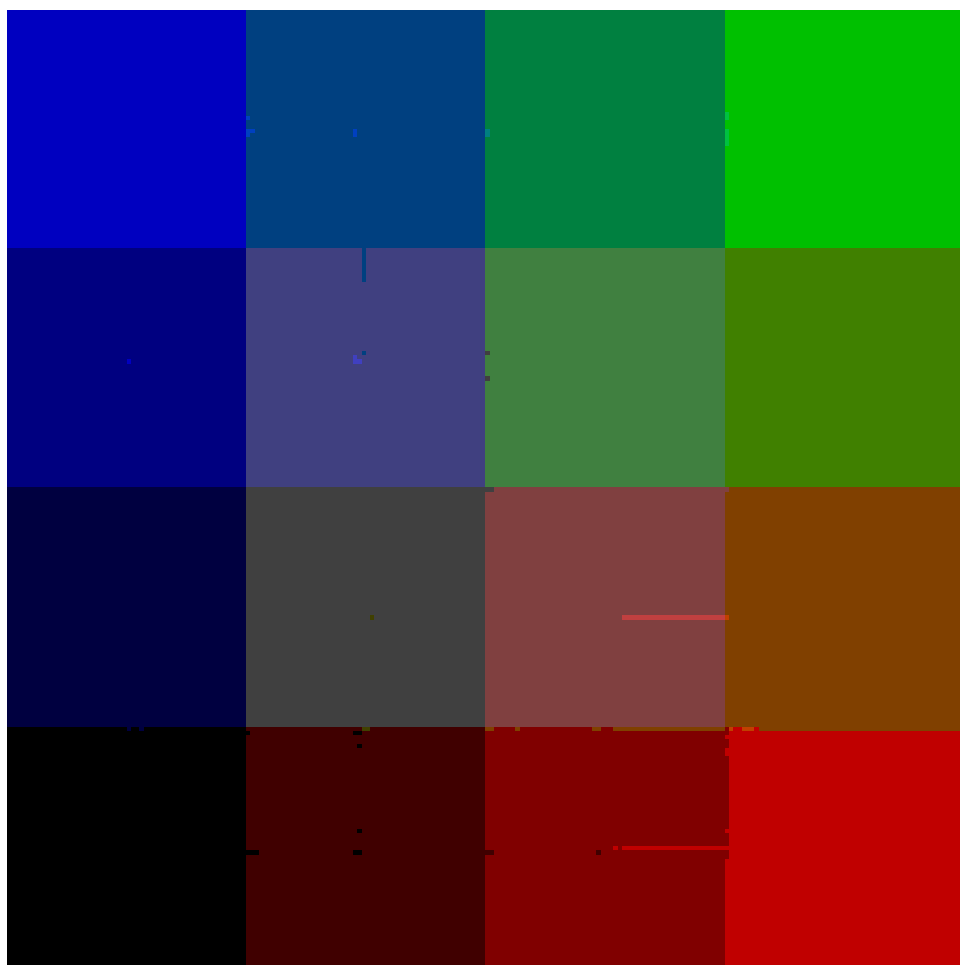
A função `BitQuantizeImage` calcula 2^k níveis e um passo de quantização de 256 dividido pelo número de níveis. O valor de cada pixel é então reduzido ao nível correspondente.

```
def BitQuantizeImage(im, k):  
    niveis = 2 ** k  
    passo = 256 // niveis  
    img_quantizada = (im // passo) * passo  
    return cv2_imshow(img_quantizada)
```

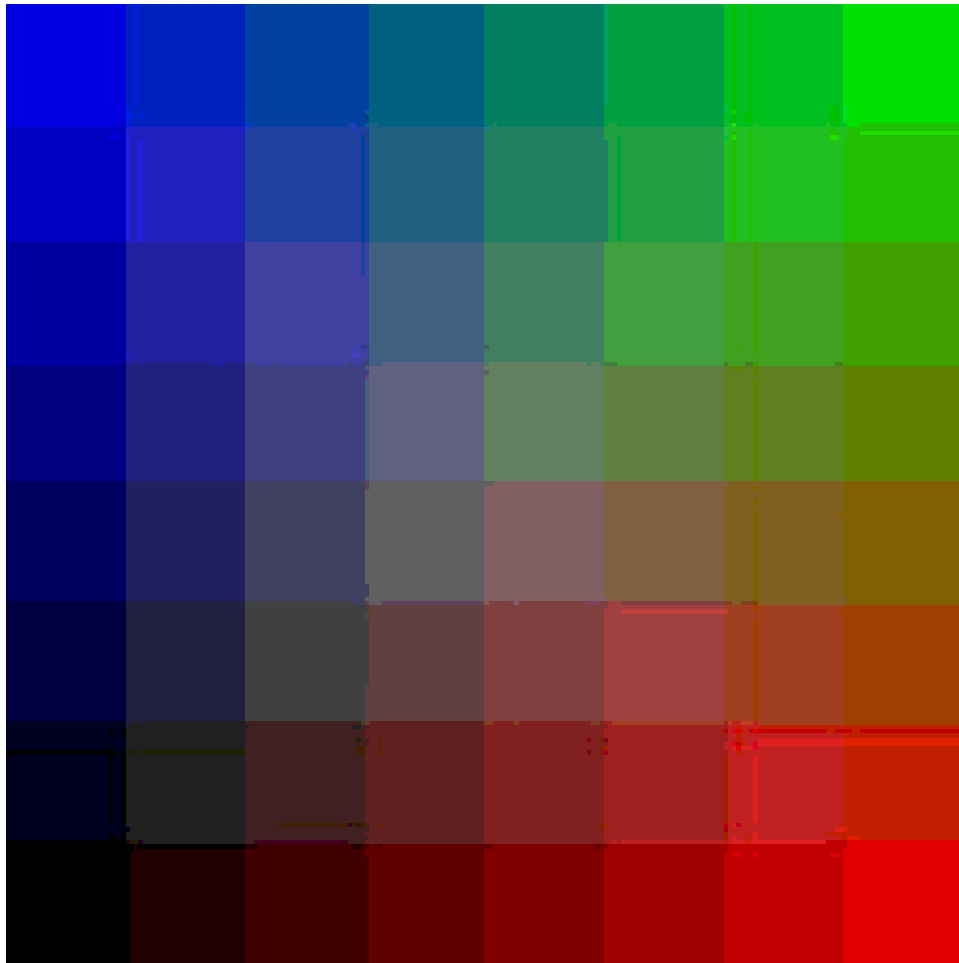
Foram realizados testes com $k=1$, $k=2$ e $k=3$, permitindo observar a perda progressiva de gradações.



Quantização com $k=1$



Quantização com $k=2$



Quantização com $k=3$

3.2.1 Planos de bits

A função `planos_bit` desloca a imagem k posições à direita e aplica uma máscara de 1 bit. O resultado é multiplicado por 255 para formar uma imagem binária visualizável.

```
def planos_bit(im, k):
    plano = (im >> k) & 1
    plano_visivel = plano * 255
    return cv2_imshow(plano_visivel)

def show_planos_bit(im, bit=8):
    for i in range(bit):
        print(f'Plano de bits {i}')
        planos_bit(im, i)
```

Os planos foram exibidos para `cameraman.png` e também para `lena.jpg`.

3.2.2 Operações identificadas em Lena

O notebook associa `lena1` à visualização do plano de bits 3, `lena2` à quantização para 2 bits e `lena3` à quantização para 1 bit.



lena1 – plano de bits 3



lena2 – quantização para 2 bits



lena3 – quantização para 1 bit

3.3 Questões diversas

3.3.1 MSB

A resposta registrada no notebook afirma que, ao remover o bit mais significativo, valores acima de 127 são removidos, resultando em um histograma limitado, de acordo com a imagem, a valores até 127.

3.3.2 LSB

A resposta registrada afirma que, ao remover o bit menos significativo, os números ímpares são removidos, fazendo o histograma apresentar lacunas nos valores correspondentes.

3.3.3 Transmissão

O notebook registra 37,45 segundos para transmitir uma imagem em tons de cinza de 512×512 pixels em um enlace de 56K baud e 0,70 segundo em um enlace de 3000K baud. Esses são os valores apresentados no material original.

O relatório mantém esses resultados como registrados, sem substituir a resposta por um novo cálculo externo, preservando a correspondência com o notebook.

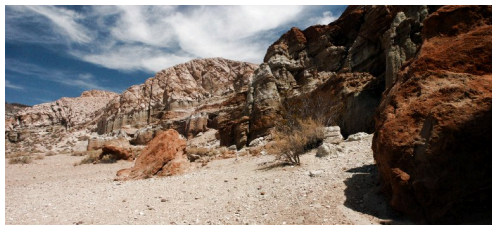
3.4 Transformação de histograma

3.4.1 Equalização

A equalização é implementada pela construção do histograma, normalização para uma PDF, cálculo da CDF e geração de um mapeamento de intensidade.

```
def equalizar_histograma(im, num_pontos, intervalo_min_max=[0, 256]):  
    point_count, point_edges = np.histogram(im, num_pontos, intervalo_min_max)  
    pdf = point_count / np.sum(point_count)  
    cdf = np.cumsum(pdf)  
    f_eq = np.round(cdf*(num_pontos-1)).astype(int)  
    im_eq = f_eq[im]  
    return np.array(im_eq, dtype=np.uint8)
```

A imagem escolhida foi canyon.png. A equalização foi aplicada uma primeira vez e novamente sobre o resultado. A observação registrada é que, quanto mais equalizada a imagem, mais a parte escura se torna visível.



canyon original



Primeira equalização



Segunda equalização

3.4.2 Especificação

A função `especificacao` calcula as CDFs de duas imagens e cria um mapeamento procurando, para cada intensidade da primeira imagem, o índice cuja CDF esteja mais próxima da segunda.

```
def especificacao(im1, im2, num_pontos=256, intervalo_min_max=[0,256]):  
    count_im1, _ = np.histogram(im1, num_pontos, intervalo_min_max)  
    pdf_im1 = count_im1 / np.sum(count_im1)  
    cdf_im1 = np.cumsum(pdf_im1)  
    count_im2, _ = np.histogram(im2, num_pontos, intervalo_min_max)  
    pdf_im2 = count_im2 / np.sum(count_im2)  
    cdf_im2 = np.cumsum(pdf_im2)  
    mapeamento = np.zeros(num_pontos, dtype=np.uint8)  
    for i in range(num_pontos):
```

```
diff = np.abs(cdf_im2 - cdf_im1[i])  
mapeamento[i] = np.argmin(diff)  
return np.array(mapeamento[im1], dtype=np.uint8)
```

Para realizar a transformação de histograma, foram selecionados três casos: imagens com histogramas semelhantes, uma transformação de uma imagem escura para uma imagem clara e uma transformação de uma imagem clara para uma imagem escura.



q7_1



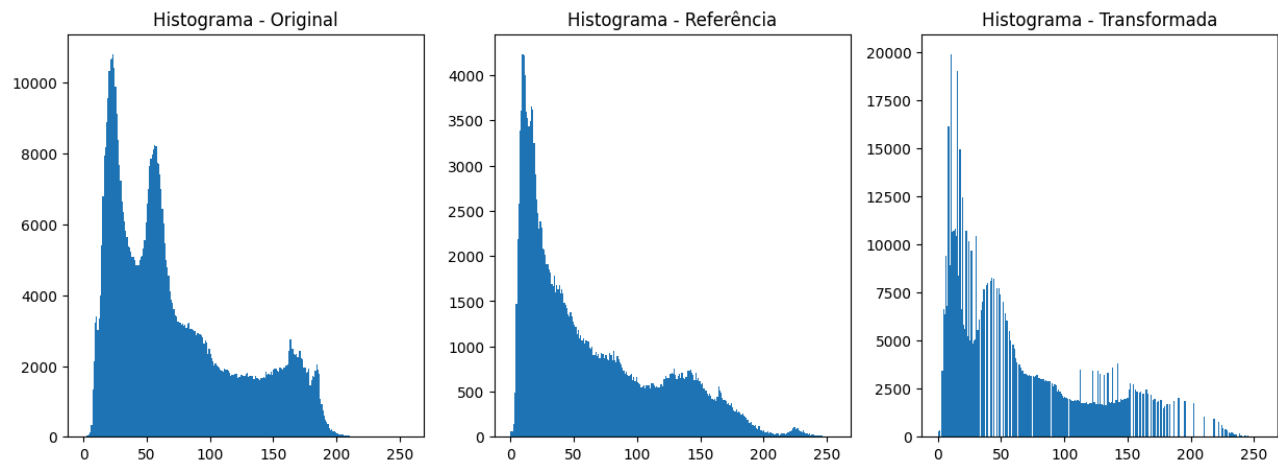
q7_2

```
def comparar_especificacao(img1, img2, resultado):
    plt.figure(figsize=(15, 5))
    plt.subplot(1, 3, 1)
    plt.imshow(img1, cmap='gray')
    plt.title("Imagem original")
    plt.axis("off")
    plt.subplot(1, 3, 2)
    plt.imshow(img2, cmap='gray')
    plt.title("Imagem de referência")
    plt.axis("off")
    plt.subplot(1, 3, 3)
    plt.imshow(resultado, cmap='gray')
    plt.title("Imagem transformada")
    plt.axis("off")
    plt.show()

def comparar_histogramas(img1, img2, resultado):
    plt.figure(figsize=(15, 5))
    plt.subplot(1, 3, 1)
    plt.hist(img1.ravel(), bins=256, range=[0, 256])
    plt.title("Histograma - Original")
    plt.subplot(1, 3, 2)
    plt.hist(img2.ravel(), bins=256, range=[0, 256])
    plt.title("Histograma - Referência")
    plt.subplot(1, 3, 3)
    plt.hist(resultado.ravel(), bins=256, range=[0, 256])
    plt.title("Histograma - Transformada")
    plt.show()
```



Comparação q7_1 e q7_2



Comparação histogramas q7_1 e q7_2

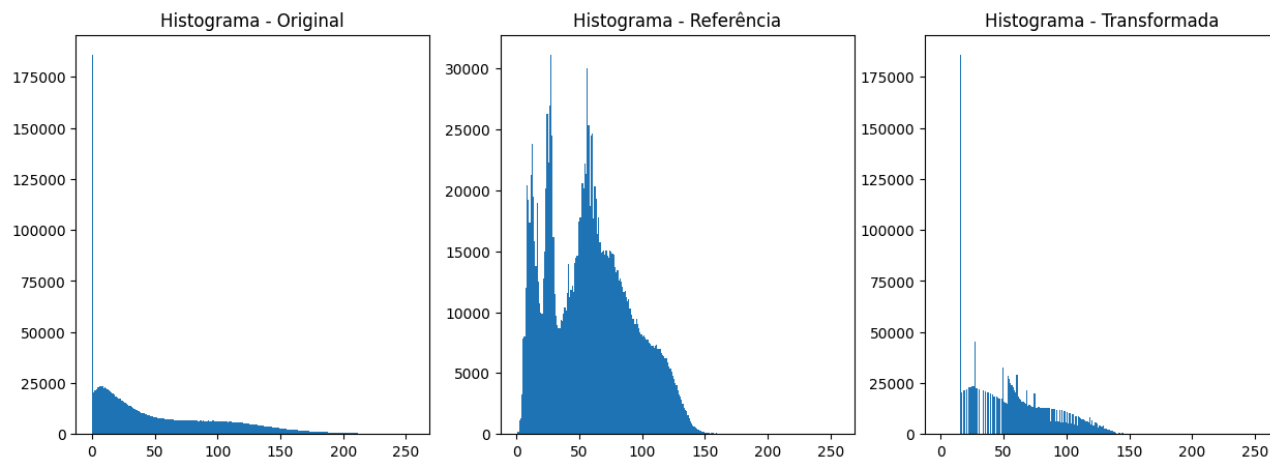
Neste primeiro caso, foram utilizadas duas imagens em escala de cinza com histogramas semelhantes. A função `especificacao()` realiza o mapeamento das intensidades da primeira imagem de acordo com a distribuição de intensidades da segunda imagem, utilizada como referência.

No segundo caso é a comparação do escuro para o claro, foram utilizados as imagens `pots_no_flash.jpg` e `pots_flash.jpg`. A transformação procura modificar a distribuição das intensidades da imagem escura para que ela se aproxime da distribuição da imagem clara.

Consequentemente, espera-se que a imagem transformada apresente um aumento nas intensidades, tornando-se visualmente mais clara e apresentando um histograma semelhante ao da imagem de referência.



Comparação pots_no_flash e pots_flash



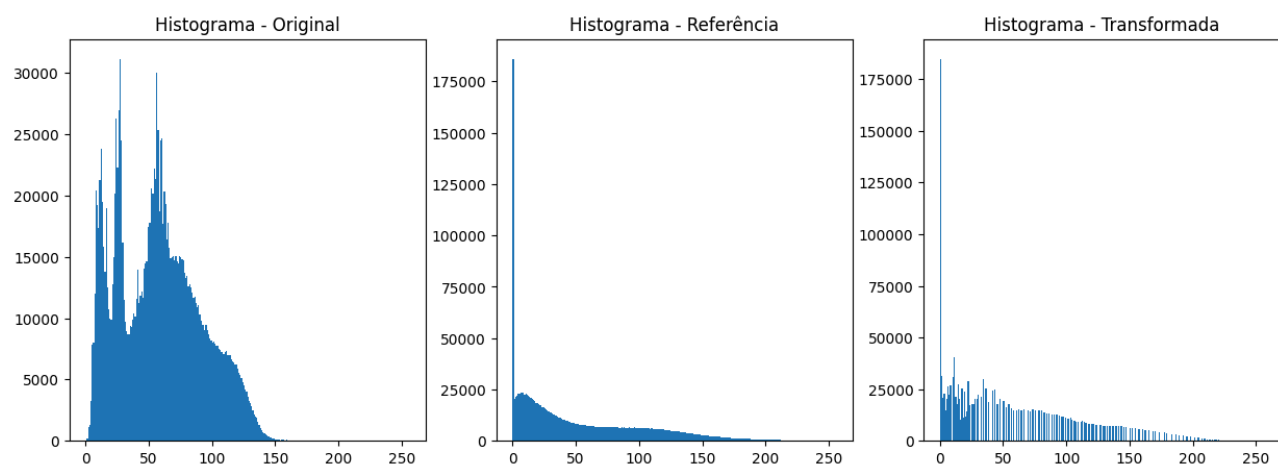
Comparação histogramas pots_no_flash e pots_flash

Neste último caso, a operação é realizada no sentido inverso. A imagem pots_flash.jpg, que apresenta maior iluminação, é utilizada como entrada, enquanto pots_no_flash.jpg é utilizada como referência.

A transformação procura deslocar a distribuição das intensidades para valores semelhantes aos da imagem escura. Dessa forma, espera-se que a imagem resultante apresente uma redução geral das intensidades, aproximando-se visualmente da imagem de referência.



Comparação pots_flash e pots_no_flash

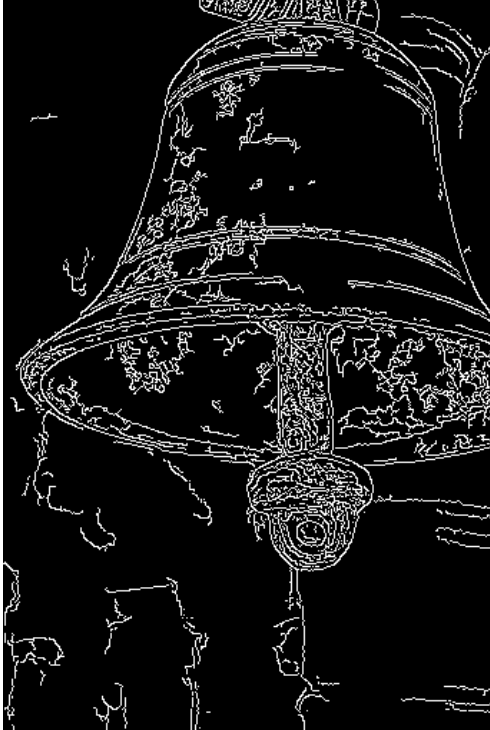


Comparação histogramas pots_flash e pots_no_flash

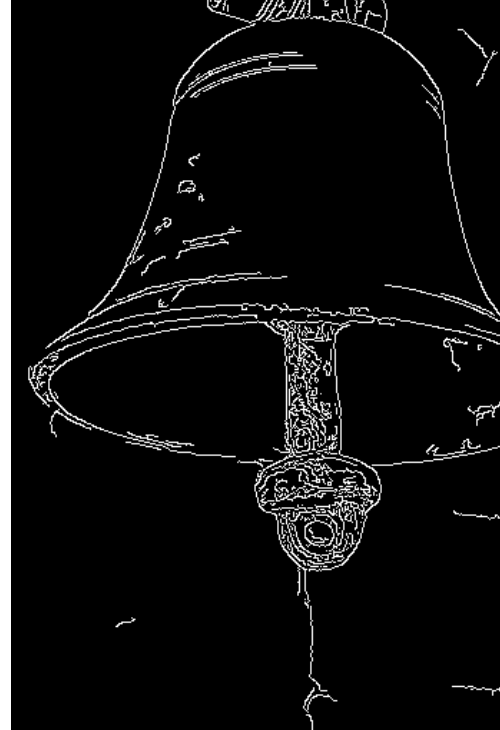
3.5 Detecção de bordas

3.5.1 Canny

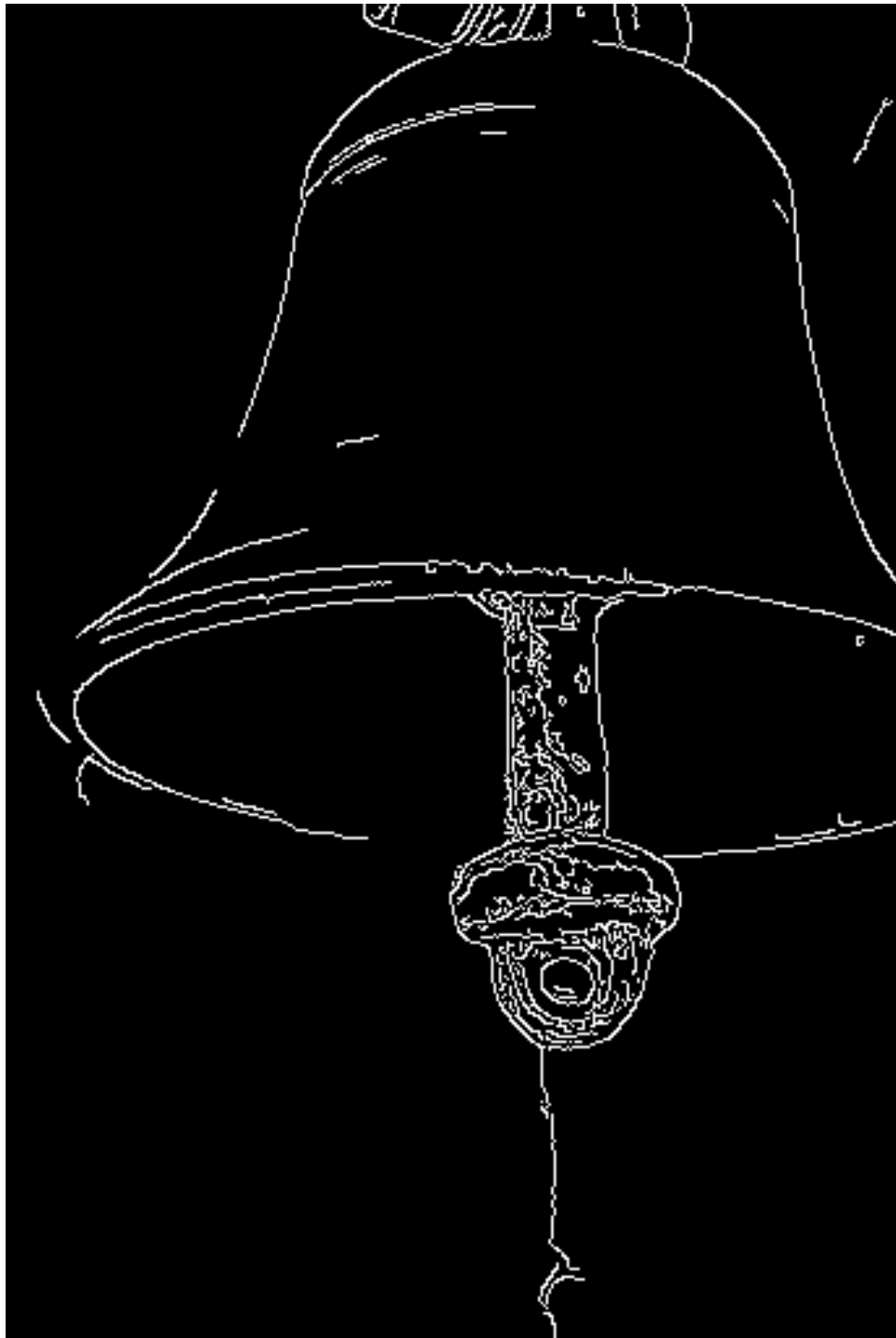
Foram testados três pares de limiares em bell.jpg: 30–90, 60–150 e 100–200. Em cubes.png foram testados 20–60, 50–150 e 80–200. O notebook apresenta as saídas, mas não registra textualmente qual combinação foi escolhida como melhor.



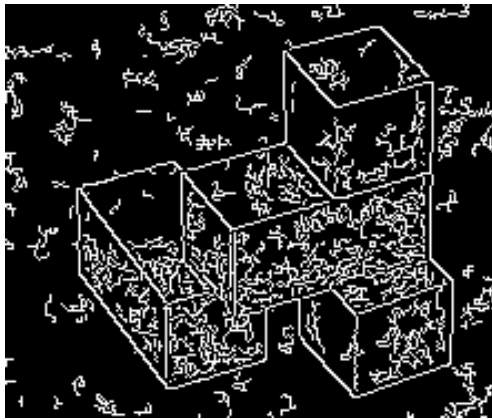
bell: 30–90



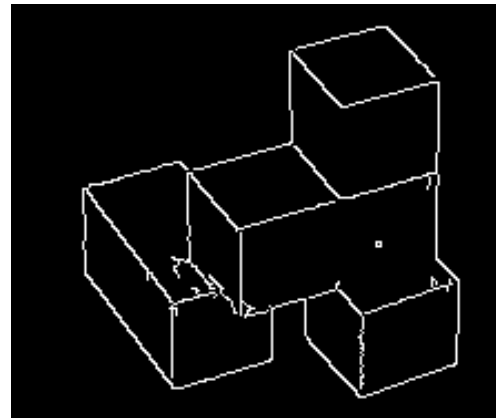
bell: 60–150



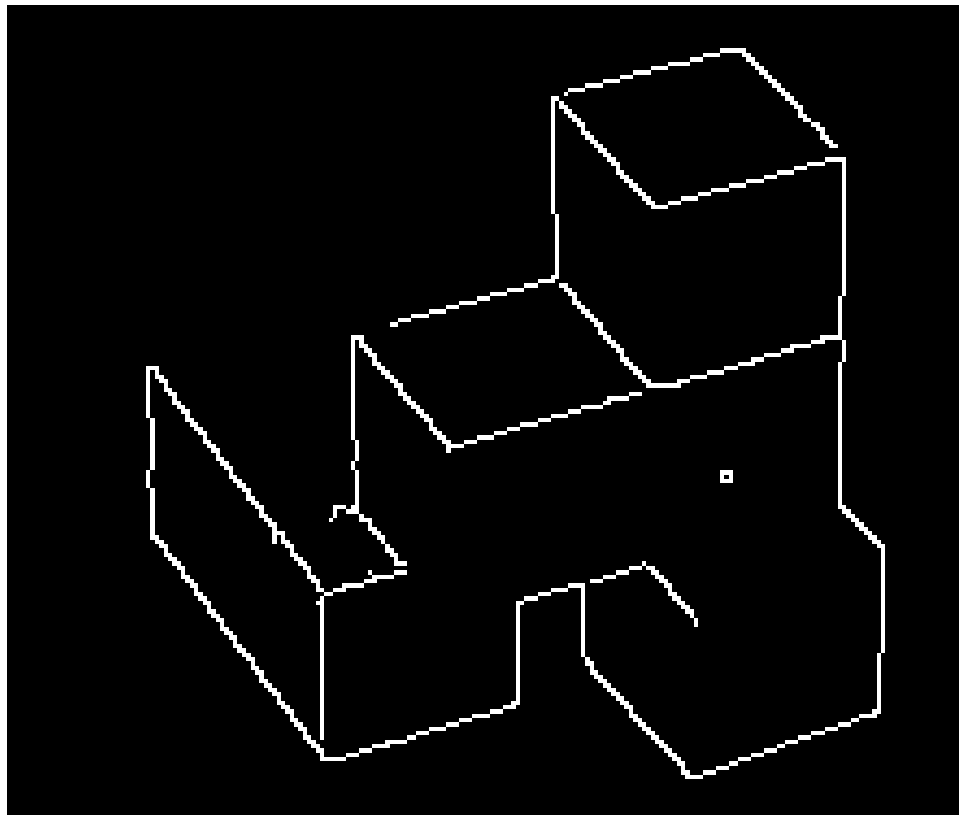
bell: 100–200



cubes: 20–60



cubes: 50–150



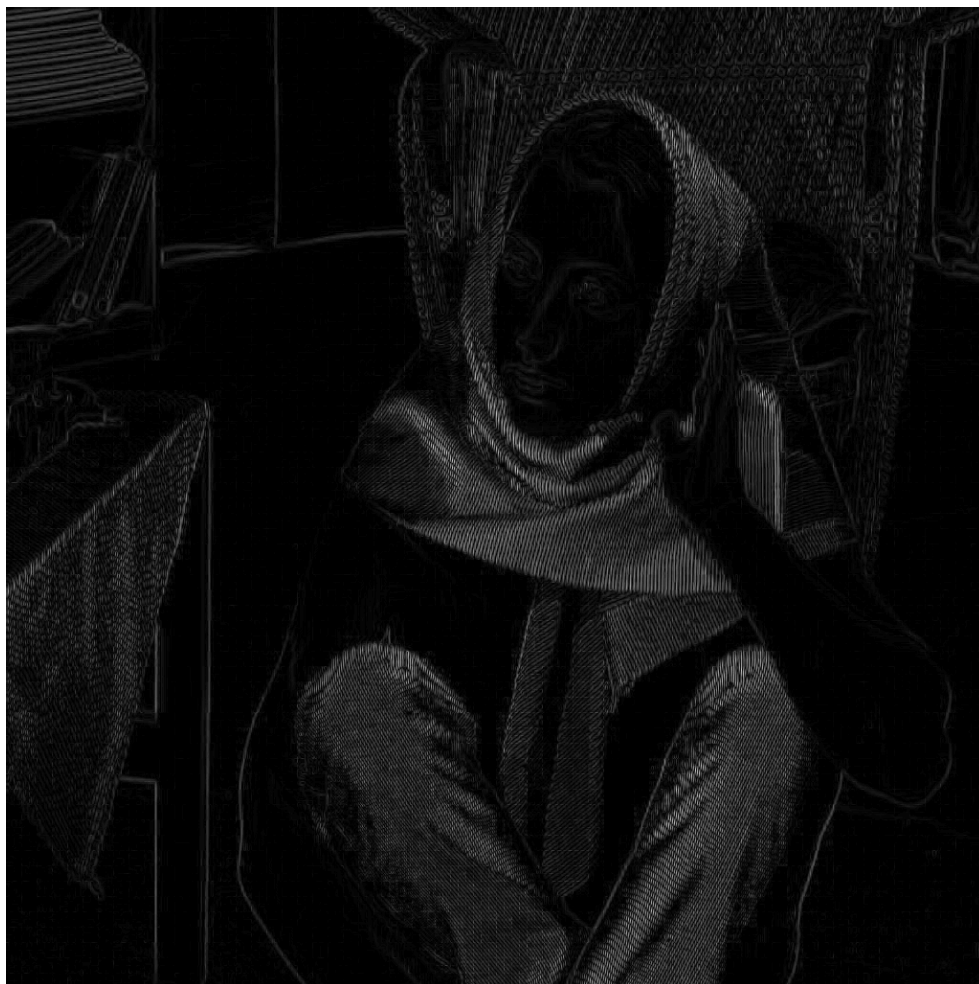
cubes: 80–200

3.5.2 Roberts, Prewitt, Sobel e Laplaciano

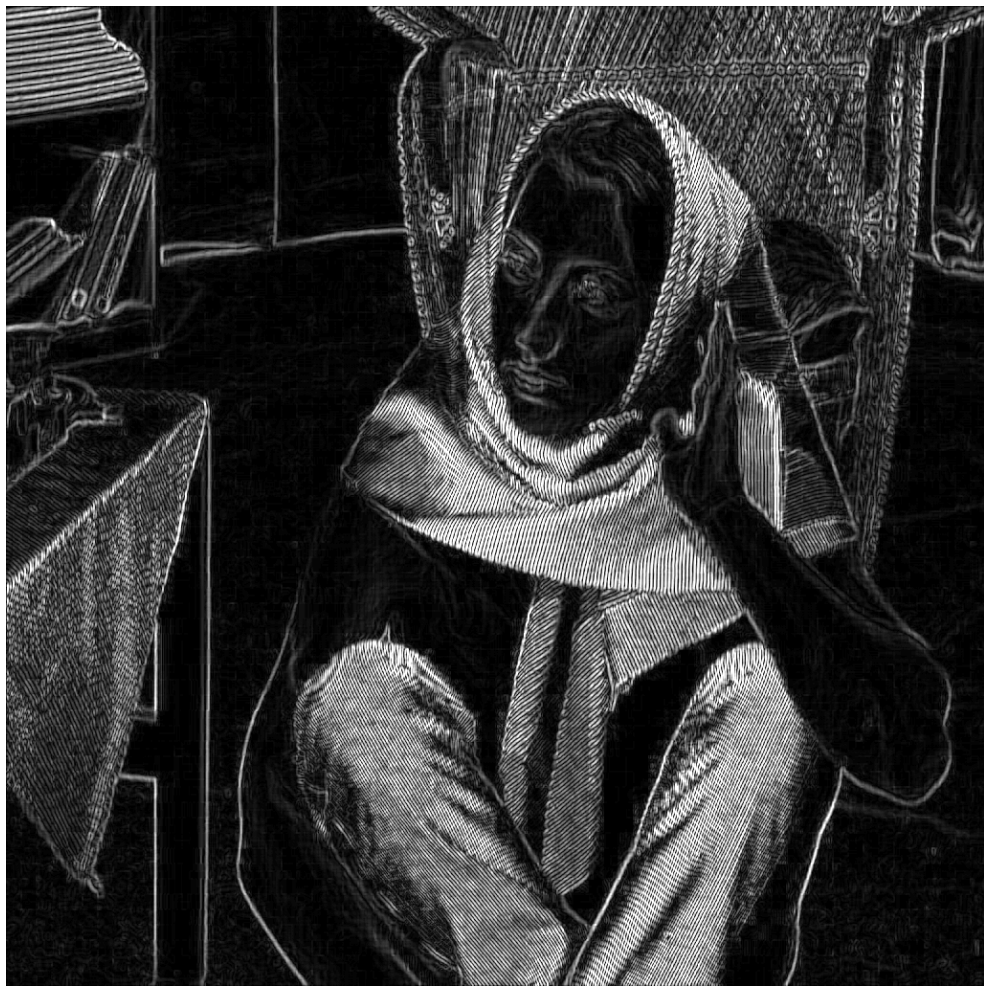
Para *barbara.jpg* foram definidos kernels de Roberts, Prewitt, Sobel e Laplaciano. Roberts, Prewitt e Sobel utilizam dois kernels direcionais, cujas respostas são combinadas pela soma dos valores absolutos.

```
kernel_laplaciano = np.array([[0,1,0],[1,-4,1],[0,1,0]])
kernel_laplaciano_8 = np.array([[1,1,1],[1,-8,1],[1,1,1]])
kernel_roberts_1 = np.array([[1,0],[0,-1]])
kernel_roberts_2 = np.array([[0,1],[-1,0]])
kernel_sobel_1 = np.array([[1,2,1],[0,0,0],[-1,-2,-1]])
kernel_sobel_2 = np.array([[-1,0,1],[-2,0,2],[-1,0,1]])
kernel_prewitt_1 = np.array([[1,1,1],[0,0,0],[-1,-1,-1]])
kernel_prewitt_2 = np.array([[-1,0,1],[-1,0,1],[-1,0,1]])
```

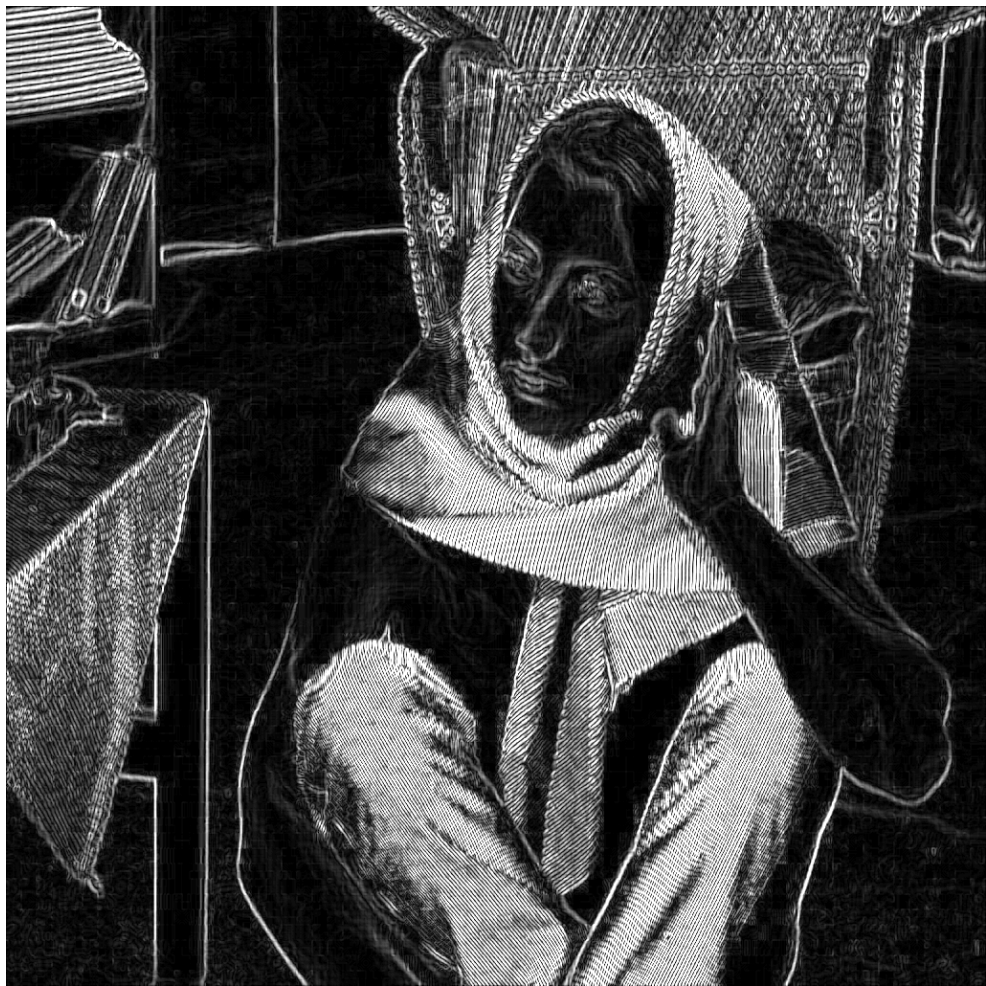
A convolução é executada manualmente pela função `conv2d_sharpening`, com padding e clipping superior.



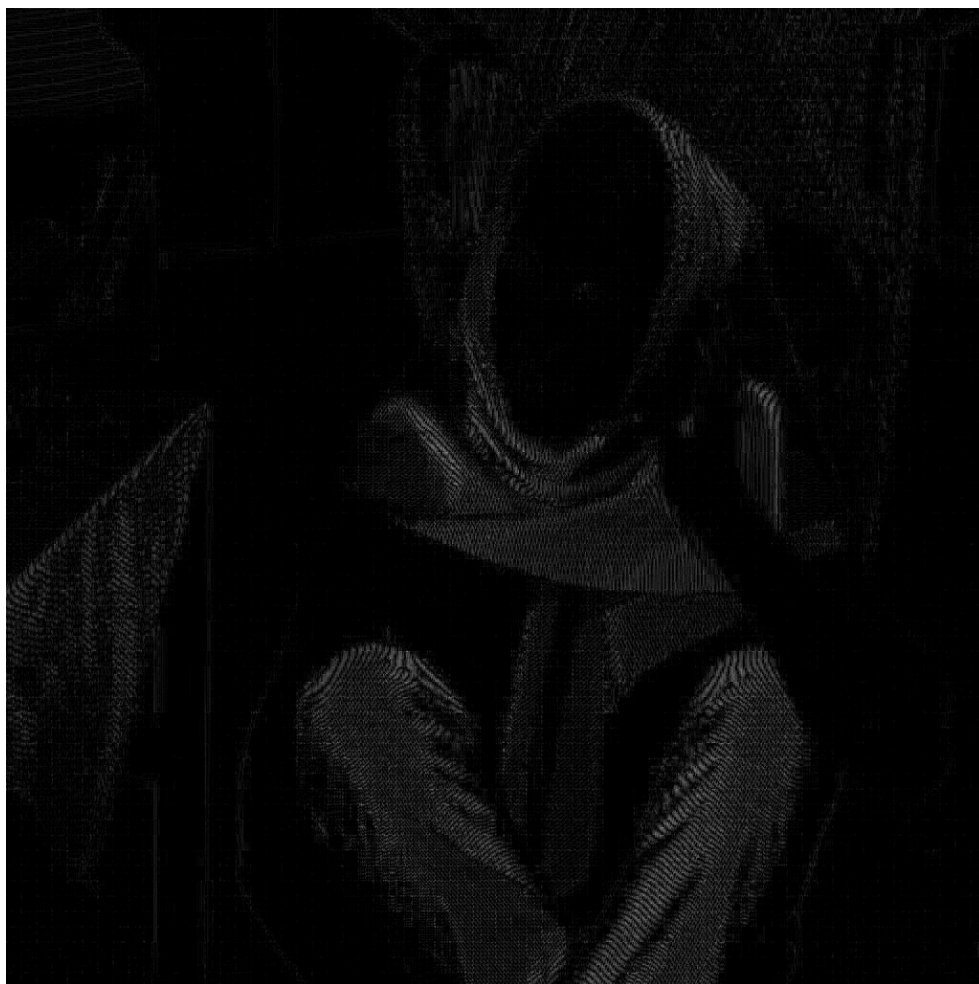
Roberts



Prewitt



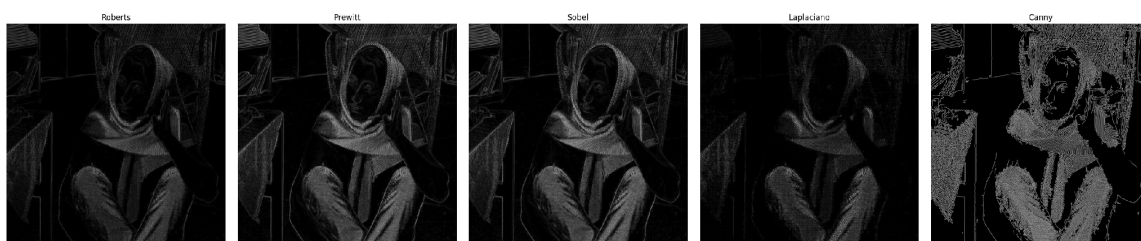
Sobel



Laplaciano



Canny em Barbara



Comparação conjunta dos cinco métodos

3.5.3 Ruído

O roteiro solicita adicionar ruído gaussiano e analisar os filtros sobre a imagem degradada. No notebook fornecido, essa parte aparece apenas como enunciado e não possui uma execução correspondente registrada. Por isso, não são atribuídos resultados inexistentes ao trabalho.

3.6 Filtragem High-Boost

O High-Boost é implementado a partir de uma imagem borrada por convolução. A máscara de alta frequência é obtida pela diferença entre a imagem original e a versão suavizada; depois, essa máscara é multiplicada por k e adicionada à imagem original.

```
def highboost(img, kernel, k):  
    blur = conv2d(img, kernel)  
    g = img.astype(float) - blur.astype(float)  
    umhf = np.zeros(img.shape, dtype=float)  
    for i in range(img.shape[0]):  
        for j in range(img.shape[1]):  
            umhf[i,j] = float(img[i,j]) + k*float(g[i,j])  
    umhf = np.clip(umhf, 0, 255)  
    return np.array(umhf, dtype=np.uint8)
```

Em bell.jpg, a imagem é convertida para tons de cinza. São usadas janelas Gaussianas 3×3 , 5×5 e 7×7 , todas com $\sigma=1$, mantendo $k=2$.



High-Boost em bell: 3×3 , 5×5 e 7×7

A observação registrada é que o aumento da janela intensifica o efeito de realce. A janela 3×3 mantém resultado mais próximo da imagem original, enquanto 5×5 e 7×7 suavizam mais agressivamente e favorecem clipping e saturação.

3.6.1 Outra imagem

A mesma análise é repetida com sky.png, convertida para escala de cinza e processada com os mesmos parâmetros.



sky original

sky em escala de cinza

Janela 3x3



Janela 5x5



Janela 7x7



High-Boost em sky

3.6.2 Bilateral versus High-Boost

A resposta do notebook diferencia os métodos pela finalidade: o Bilateral usa intensidade e distância para decidir onde suavizar, preservando bordas; o High-Boost amplifica a diferença entre a imagem original e uma versão borrada.

4. CÓDIGOS IMPORTANTES DA IMPLEMENTAÇÃO

Os trechos abaixo representam as estruturas centrais que sustentam os experimentos.

4.1 Convolução manual

```
for i_img in range(img_height):
    for j_img in range(img_width):
        for i_kernel in range(k_height):
            for j_kernel in range(k_width):
                output[i_img, j_img] += (
                    float(padded_img[i_img+i_kernel, j_img+j_kernel])
                    * kernel[i_kernel, j_kernel]
                )
```

4.2 Kernel Gaussiano

```
def gauss_create(sigma=1, size_x=3, size_y=3):
    x, y = np.meshgrid(
        np.linspace(-1,1,size_x),
        np.linspace(-1,1,size_y))
    calc = 1/((2*np.pi*(sigma**2)))
    exp = np.exp(-(((x**2)+(y**2))/(2*(sigma**2))))
    return exp*calc
```

4.3 Conversão para grayscale

```
B, G, R = cv2.split(bell)
bell_gray = 0.299*B + 0.587*G + 0.114*R
bell_gray = np.array(bell_gray, dtype=np.uint8)
```

4.4 Padding

```
def add_padding(img, padding_height, padding_width):
    n, m = img.shape
    padded_img = np.zeros(
        (n + padding_height * 2, m + padding_width * 2))
    padded_img[
        padding_height:n + padding_height,
        padding_width:m + padding_width
    ] = img
    return padded_img
```

Essas funções evidenciam a preocupação do trabalho em construir operações de baixo nível com NumPy e loops explícitos, especialmente para convolução e filtragem.

5. ANÁLISE E DISCUSSÃO SOBRE OS RESULTADOS FINAIS

A análise conjunta mostra que as técnicas possuem objetivos distintos. A manipulação de pixels atua diretamente nos valores de cor; a manipulação de bits controla a informação binária; a transformação de histograma altera a distribuição das intensidades; os detectores enfatizam transições; High-Boost realça altas frequências; e Bilateral procura suavizar preservando diferenças importantes.

5.1 Pixels

A estratégia de usar a cor mais frequente é simples e permite construir uma máscara sem depender de segmentação complexa. O parâmetro tolerance torna o método mais permissivo a variações de cor.

5.2 Bits

A quantização demonstra diretamente a relação entre bits e níveis de intensidade. A redução para 1, 2 ou 3 bits produz uma diminuição progressiva das gradações. A visualização dos planos complementa essa análise.

5.3 Histogramas

A equalização baseada em CDF modifica a distribuição de intensidades e pode tornar regiões escuras mais perceptíveis, como observado pelo autor no notebook. A segunda equalização demonstra que uma imagem já transformada ainda pode sofrer novas alterações.

5.4 Bordas

Roberts trabalha com kernels 2×2 , enquanto Prewitt e Sobel usam kernels 3×3 . Sobel atribui maior peso às posições próximas ao centro. O Laplaciano representa uma resposta de segunda derivada. Canny apresenta limiares ajustáveis e, no notebook, foi comparado por diferentes pares de valores.

5.5 High-Boost

O efeito do High-Boost depende diretamente do borramento utilizado como referência. Janelas maiores alteram mais intensamente as componentes de alta frequência e podem produzir clipping, como observado nas imagens do notebook.

6. CONSIDERAÇÕES FINAIS

O notebook apresenta uma implementação abrangente de conceitos fundamentais de Processamento de Imagens, cobrindo desde operações diretamente aplicadas aos pixels até filtros espaciais, transformações de histogramas e detectores de bordas.

A construção manual de convolução, kernels direcionais, High-Boost e Bilateral é um dos aspectos centrais do trabalho, pois permite observar a sequência de cálculos que produz os resultados. A manipulação de bits e a quantização, por sua vez, evidenciam a influência da representação digital sobre a aparência da imagem.

Os experimentos também mostraram que não existe uma única transformação adequada para todos os objetivos. Equalização, detecção, realce e suavização respondem a necessidades diferentes e devem ser avaliados de acordo com a finalidade da imagem.

Houve certa dificuldade nos resultados da filtragem Bilateral, e devido a resultados considerados não suficientes, foram removidos do documento.

Por fim, algumas atividades permanecem incompletas no notebook ou possuem elementos que impedem uma conclusão totalmente documentada, como a etapa de ruído, a especificação de histograma e a variável `noir_gt`. Essas limitações foram mantidas explicitamente para que este relatório representasse fielmente o arquivo analisado.